

Cuestiones

Por Pedro Ismael Haddou y Antonio Moroño.

Apartado 3

Ejecución del Programa con Distintos Tipos de Parámetros

1. Sin parámetros:

- Mensaje de error: "Se espera al menos una palabra como parámetro."

2. Con palabras como parámetros:

- Ejemplo: `java LongitudPalabras coche esta guapo`
- Resultado en la terminal:

```
Las longitudes de las palabras son:  
- coche (5 caracteres).  
- esta (4 caracteres).  
- guapo (5 caracteres).  
Palabras almacenadas: [coche, esta, guapo]  
Longitud de 'No_almacenada': -1
```

- Observaciones:
 - El programa funciona correctamente, almacenando las palabras introducidas.
 - La longitud de la palabra "No_almacenada" es -1 porque no está en la colección.

3. Con la palabra "No_almacenada" como parámetro:

- Ejemplo: `java LongitudPalabras coche esta guapo No_almacenada`
- Resultado en la terminal:

```
Las longitudes de las palabras son:  
- coche (5 caracteres).  
- esta (4 caracteres).  
- guapo (5 caracteres).  
- No_almacenada (13 caracteres).  
Palabras almacenadas: [coche, esta, guapo, No_almacenada]  
Longitud de 'No_almacenada': 13
```

- Observaciones:
 - Al ejecutar el programa con "No_almacenada" como parámetro, la palabra se almacena y su longitud se muestra correctamente.

4. Con un número como parámetro:

- El número se convierte a cadena de caracteres (String).

Notas Adicionales:

- El programa utiliza un mapa para almacenar las palabras y sus longitudes.
- La longitud de una palabra no almacenada es -1.
- El programa muestra mensajes de error y resultados esperados según la situación.

Apartado 4

Cálculo de Frecuencias y Alternativas

1. Estrategia de Cálculo de Frecuencias:

- Método `getFrecuencia` recorre la lista de palabras.
- Cada vez que la longitud de una palabra coincide con la entrada, se aumenta en 1 la frecuencia de esa longitud.
- Método `getLongitudesUnicas` devuelve el conjunto de longitudes únicas.

2. Alternativa con HashMap:

- Utilizar un HashMap (clave: longitud, valor: frecuencia) para calcular frecuencias.
- Ventajas:
 - Recorrer la lista de palabras solo una vez.
 - Acceso rápido a la frecuencia de una longitud específica.
- Desventajas:
 - Puede requerir más memoria, especialmente con listas grandes.
 - Rendimiento afectado al actualizar el HashMap en cada iteración.

Implementación del Método `getLongitudesRepetidas`

```
public HashSet<Integer> getLongitudesRepetidas() {
    HashSet<Integer> setUnicas = new HashSet<Integer>();
    HashSet<Integer> setRepetidas = new HashSet<Integer>();
    for (Integer valor : this.palabras.values()) {
        if (setUnicas.contains(valor)) {
            setRepetidas.add(valor);
        } else {
            setUnicas.add(valor);
        }
    }
    return setRepetidas;
}
```

- Este método devuelve las longitudes repetidas basándose en la comparación con el HashSet de longitudes únicas.
- Ejemplo: Con las palabras "codigo casa gato", la salida sería el conjunto {4} ya que 4 es la única longitud que se repite.